



RetroC

Retro CPU Simulator Compiler

Compiler Construction CS4031

Submitted to:

Mr. Zulfiqar Ali

Submitted by:

Amna Khan 23K-0856

Kissa Zehra 23K-0706

Afaq Ahmed 23K-0901

Section:

BCS-6B

1. Project Introduction

RetroC is a statically-typed, block-scoped, imperative language modelled after early C dialects, designed to feel like programming on vintage computing hardware. Source files carry the `.rc` extension. The compiler translates source code into Three-Address Code (TAC), the custom assembly of our simulated Retro CPU, which is then executed by a built-in virtual machine simulating the fetch-decode-execute cycle. The CLI supports file compilation, output redirection with the `-o` flag, a debug VM tracer, `--ir` TAC dump, and an interactive REPL for live testing.

Phase 1) Lexical Analysis: The hand-written scanner (`scanner.cpp` / `scanner.h`) tokenises source code using a DFA, tracking line numbers at every token to produce precise error messages.

Phase 2) Syntax Analysis: A recursive-descent parser (`parser.cpp` / `parser.h`) translates the full RetroC EBNF grammar into a working implementation using predictive parsing and single-token lookahead.

Phase 3) Semantic Analysis: The semantic analyser (`semantic.cpp` / `semantic.h`) enforces a scoped symbol table, type checking, function signature validation, and use-before-initialisation detection.

Phase 4) IR Generation: The IR module (`ir.cpp` / `ir.h`) emits Three-Address Code instructions in a single pass, generating fresh temporaries (`_t0`, `_t1`, ...) and dynamic labels (`_L0`, `_L1`, ...) on the fly.

Phase 5) Optimization: The optimizer (`optimizer.cpp` / `optimizer.h`) performs Constant Folding and Dead Code Elimination directly over the TAC instruction list without a separate data-flow framework.

Phase 6) Code Generation & VM: The interpreter (`interpreter.cpp` / `interpreter.h`) executes TAC via a call stack of activation records, implementing a return-address mechanism for nested and recursive calls.

2. Challenges Faced & Solutions

Building RetroC end-to-end exposed us to the full complexity of compiler engineering. Below are the key technical challenges we encountered and the strategies we applied to resolve them.

- 1. Scanner & Line Tracking:** Our initial scanner had no line counter, making every error message blind. We threaded a line counter through the entire scanning process and attached the current line number to every token at creation, a simple fix with a profound impact on all downstream diagnostics.
- 2. Two-Character Operator Lookahead:** Correctly recognising `<=`, `>=`, `==`, `!=`, `&&`, and `||` required a one-character lookahead strategy: after consuming the first character, we peek at the next before committing to a token type. Edge cases like a lone `!` not followed by `=` required careful handling.
- 3. For-Loop IR Buffering:** The for-loop increment must appear textually before the body but execute after it. We solved this by recording the IR list size before parsing the increment, extracting those instructions into a temporary buffer, parsing the body, then re-appending the buffer, and correcting the execution order without an AST.
- 4. Statement Disambiguation:** When the parser sees an identifier at statement start, a single-token lookahead resolves the ambiguity: `T_ASSIGN` means variable assignment, `T_LPAREN` means function call, and `T_LBRACKET` means array element assignment.

5. **Sentinel Unknown Types:** Without precautions, an undeclared variable would trigger cascading errors on every subsequent operation. We introduced an 'unknown' sentinel return type: the error is reported once, and all downstream checks silently skip any operand typed as unknown.
6. **Dynamic Label Generation:** For if/else and loop control flow we need labels whose values are unknown when the condition is first parsed. We generate fresh names with a monotonically incrementing counter and capture them by value immediately, before subsequent instructions can overwrite local variables.
7. **Dead Code Reachability:** Our dead-code elimination pass tracks a reachable flag. An unconditional RETURN or GOTO sets it to false; any instruction encountered while false is marked dead. Crucially, a LABEL instruction must always reset the flag to true, because a jump from elsewhere can transfer control to that label.
8. **VM Return-Address Stack:** Each activation record stores the index of the instruction immediately following the corresponding CALL. On RETURN, the VM reads this saved index and jumps back, cleanly handling arbitrarily nested and recursive function calls.

3. What We Learned About Compiler Design

The following are the most significant lessons we took away from building RetroC end to end.

1. **Compilers are precisely layered systems:** Every design decision in one phase creates constraints in all subsequent phases. Adding line numbers to the scanner was not merely a quality-of-life feature, it fundamentally changed how informative and usable the entire compiler was for anyone debugging their code.
2. **Three-Address Code is an extraordinarily elegant IR:** By reducing all computation to simple three-operand instructions, TAC is simultaneously easy to generate from recursive descent parsing, easy to optimise by pattern-matching over instruction sequences, and easy to execute by dispatching on opcode in the VM.
3. **Good error recovery requires as much design effort as the happy path:** Writing a compiler that crashes on the first error is easy. Writing one that produces a single precise, actionable message and continues cleanly is genuinely hard, it demands careful architectural decisions, not just extra code.
4. **Optimisation requires a global view of the program:** Our constant folding pass works because it examines the entire instruction list, not just one instruction in isolation. This gave us concrete intuition for why more powerful optimisations, such as common subexpression elimination or loop-invariant code motion, require full data-flow analysis over the control-flow graph.

4. Future Improvements

Given additional time and resources, the following enhancements would make RetroC significantly more powerful and bring it closer to the design of real production compilers.

1. **While-Loop Support:** The current language supports only for-loops. Adding a while keyword and its corresponding parser rule would be the most impactful language extension, as many natural algorithms are more clearly expressed with a while loop.

2. **Richer Type System:** RetroC currently supports only int, bool, and char. Adding a float type with appropriate arithmetic rules, and string literals as a first-class type, would make the language significantly more practical for real programs.
3. **Better Error Recovery:** Our panic-mode recovery synchronises to the nearest semicolon on a syntax error. A more sophisticated phrase-level recovery strategy would allow the compiler to detect and report multiple independent errors in a single pass, greatly improving the developer experience.
4. **Register Allocation:** Our VM uses symbolic variable names directly. Implementing a linear-scan register allocator that maps temporaries to a fixed set of virtual registers would bring RetroC much closer to how real compiler back-ends work.
5. **Real Assembly Code Generation:** Currently RetroC's output is the interpreted result of executing TAC on our VM. A natural extension would be a code generation back-end that emits actual assembly, targeting a simple RISC instruction set, so compiled programs could run natively without the interpreter.

5. Individual Contributions

The project was divided across three core pipeline areas, with each member taking primary ownership of two compiler phases and their associated deliverables. All members participated in language design, testing, and integration.

Member	ID	Primary Contributions
Amna Khan	23K-0856	Lexical Analyzer (scanner.cpp/h), IR Generation (ir.cpp/h), -o CLI flag, Test Programs 2 & 4, integration testing, README
Kissa Zehra	23K-0706	Recursive Descent Parser (parser.cpp/h), Optimizer — Constant Folding & Dead Code Elimination, Test Program 3, parse tree docs
Afaq Ahmed	23K-0901	Semantic Analyzer (semantic.cpp/h), Virtual Machine/Interpreter (interpreter.cpp/h), Language Reference Manual, Test Programs 1 & 5